

Exercício – Aula 16 PHP + BD

Sistema de Gestão de Hotel/Pousada

PostgreSQL • PHP • PDO • POO • CRUD

1. Objetivo

Desenvolver individualmente o backend de um sistema de gestão de hotel/pousada, utilizando PostgreSQL, PHP, PDO, Programação Orientada a Objetos e operações CRUD. O exercício integra modelagem de banco de dados, SQL, conexão com banco, classes e processamento de formulários.

O front-end não será desenvolvido pelo aluno. Será disponibilizada uma base em HTML, CSS e Bootstrap para ser integrada ao backend.

2. Tecnologias obrigatórias

- PostgreSQL
- PHP
- PDO com o driver PostgreSQL (pdo_pgsql)
- Programação Orientada a Objetos
- Git

HTML, CSS e Bootstrap serão utilizados a partir do front-end fornecido.

3. Contexto

Uma pousada deseja informatizar o controle de hóspedes, quartos, reservas, serviços adicionais e pagamentos. O sistema deve permitir cadastrar e consultar informações, alterar registros, excluir dados quando permitido e gerar consultas para apoiar a gestão da pousada.

4. Regras de negócio

Hóspedes

- Armazenar nome, CPF, telefone e e-mail.
- Um hóspede pode realizar várias reservas.
- O CPF deve ser único.

Tipos de quarto

- A pousada possui diferentes tipos, como Individual, Duplo, Triplo e Suíte.
- Cada tipo possui nome e descrição.
- Um tipo pode estar associado a vários quartos.

Quartos

- Cada quarto possui número, tipo, capacidade, valor da diária e status.
- O número deve ser único.
- O status deve representar situações como disponível, ocupado e manutenção.

Reservas

- Registrar hóspede, quarto, data de entrada, data de saída, quantidade de hóspedes, status e observação.
- Uma reserva pertence a um hóspede; um hóspede pode possuir várias reservas.
- Um quarto pode possuir várias reservas ao longo do tempo.
- O sistema deve evitar reservas conflitantes para o mesmo quarto.
- Status mínimos: Reservada, Hospedado, Finalizada e Cancelada.

Serviços

- Exemplos: café da manhã, estacionamento, lavanderia, traslado, passeio turístico e serviço de quarto.
- Cada serviço possui nome, descrição, preço e situação ativo/inativo.
- Uma reserva pode utilizar vários serviços e um serviço pode estar associado a várias reservas.
- Deve ser registrada a quantidade utilizada de cada serviço.

Pagamentos

- Uma reserva pode possuir pagamentos.
- Registrar reserva, valor, data, forma de pagamento e situação.
- Exemplos de formas: dinheiro, cartão de crédito, cartão de débito e PIX.

5. Modelo Entidade-Relacionamento

Analise as regras de negócio e construa o modelo ER. O modelo não será fornecido pronto.

- entidades e atributos
- chaves primárias
- chaves estrangeiras
- relacionamentos
- cardinalidades
- entidades associativas, quando necessárias

O ER deverá ser salvo no Git, por exemplo em modelo/er.pdf ou modelo/er.png.

6. Banco de dados

Criar banco/01_criacao.sql, capaz de criar as tabelas a partir do zero.

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- UNIQUE
- DEFAULT
- CHECK

Defina corretamente a integridade referencial.

7. Massa de dados

Criar banco/02_dados.sql com dados suficientes para testar o sistema.

- mínimo de 10 hóspedes
- mínimo de 8 quartos
- mínimo de 4 tipos de quarto
- mínimo de 15 reservas
- mínimo de 6 serviços
- registros de utilização de serviços
- pagamentos

Inclua situações variadas: reservas futuras, finalizadas e canceladas; hóspedes com várias reservas; diferentes serviços e formas de pagamento.

8. Consultas SQL

Criar banco/03_consultas.sql.

Reservas por período

Apresentar hóspede, quarto, tipo do quarto, entrada, saída e status, permitindo filtrar o período.

Quartos ocupados

Apresentar os quartos ocupados em determinada data, com quarto, tipo, hóspede, entrada e saída.

Faturamento por tipo de quarto

Apresentar tipo de quarto, quantidade de reservas, quantidade de diárias e faturamento, ordenando pelo maior faturamento.

Serviços mais utilizados

Apresentar serviço, quantidade de utilizações, quantidade total e valor gerado.

Hóspedes que mais gastaram

Apresentar os 10 hóspedes com maior gasto, incluindo quantidade de reservas, hospedagem, serviços e total.

Quartos sem reservas

Listar quartos que nunca tiveram reserva.

Reservas por status

Apresentar cada status e a quantidade de reservas.

As consultas deverão incluir, entre o conjunto entregue: pelo menos 3 JOIN; 1 LEFT JOIN; 2 GROUP BY; COUNT; SUM; ORDER BY; WHERE; filtro por datas; HAVING; LIMIT.

9. Conexão com PostgreSQL utilizando PDO

Neste exercício NÃO deverá ser utilizada `pg_connect()`. Na atividade anterior foi utilizada a conexão nativa do PostgreSQL. Agora você deverá pesquisar e utilizar PDO com PostgreSQL.

Pesquise na documentação do PHP por:

- PDO PostgreSQL
- `pdo_pgsql`
- DSN
- `PDO::prepare()`
- `PDOStatement::execute()`
- `fetch()`
- `fetchAll()`

A conexão utiliza uma DSN do PostgreSQL. Exemplo conceitual:

```
$pdo = new PDO("pgsql:host=localhost;port=5432;dbname=nome_do_banco",  
$usuario, $senha);
```

Não copie o exemplo sem compreender. Pesquise como configurar tratamento de erros e como estruturar uma classe responsável pela conexão.

Dados recebidos dos formulários devem utilizar consultas parametrizadas:

```
$stmt = $pdo->prepare("SELECT * FROM hospedes WHERE id = :id");  
$stmt->execute(["id" => $id]);
```

10. Classe de conexão

Crie uma classe responsável pela conexão, por exemplo:

```
config/Database.php
```

A classe deverá centralizar a criação/obtenção da conexão PDO.

11. Programação Orientada a Objetos

O backend deverá utilizar classes. Não desenvolva o sistema apenas como um conjunto de scripts procedurais.

```
classes/  
├── Hospede.php  
├── Quarto.php  
├── TipoQuarto.php  
├── Servico.php  
├── Reserva.php  
└── Pagamento.php
```

Exemplo de classe — Hospede

A classe poderá possuir os seguintes atributos:

```
class Hospede  
{  
    private int $id;  
    private string $nome;  
    private string $cpf;  
    private string $telefone;  
    private string $email;  
}
```

E métodos relacionados às operações necessárias:

```
public function __construct(...)  
public function cadastrar(PDO $pdo): bool  
public static function listar(PDO $pdo): array  
public static function buscarPorId(PDO $pdo, int $id): ?Hospede  
public function atualizar(PDO $pdo): bool  
public function excluir(PDO $pdo): bool
```

Esse é apenas um exemplo de estrutura. Você deverá implementar os métodos e decidir como organizar as demais classes, atributos, parâmetros e retornos.

12. CRUD obrigatório

Hóspedes

Implementar CRUD completo: CREATE, READ, UPDATE e DELETE.

Quartos

Implementar CRUD completo: CREATE, READ, UPDATE e DELETE.

Serviços

Implementar CRUD completo: CREATE, READ, UPDATE e DELETE.

Reservas

Implementar cadastro, listagem, alteração e exclusão de reservas. Os campos de hóspede e quarto deverão ser carregados do banco em elementos de seleção.

13. Front-end fornecido

Será fornecida uma base em HTML, CSS e Bootstrap com páginas para início, hóspedes, quartos, serviços e reservas. Os dados exibidos inicialmente são exemplos. O aluno deverá integrar os formulários e listagens ao próprio backend.

O front-end não será avaliado como desenvolvimento visual. A avaliação concentra-se no banco e no backend.

14. Organização sugerida do Git

```
hotel-pousada/  
├── README.md  
├── banco/  
│   ├── 01_criacao.sql  
│   ├── 02_dados.sql  
│   └── 03_consultas.sql  
├── modelo/  
│   └── er.pdf  
├── config/  
│   └── Database.php  
├── classes/  
│   ├── Hospede.php  
│   ├── Quarto.php  
│   ├── TipoQuarto.php  
│   ├── Servico.php  
│   ├── Reserva.php  
│   └── Pagamento.php  
└── demais arquivos PHP
```

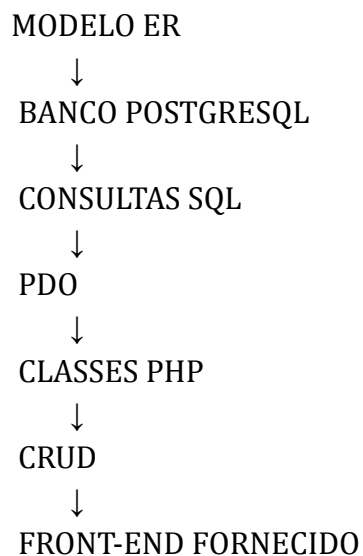
A estrutura acima é uma sugestão. Outra organização é permitida, desde que o repositório esteja claro e organizado.

15. Entrega no Git

- README.md
- modelo ER
- 01_criacao.sql
- 02_dados.sql
- 03_consultas.sql
- código PHP
- classes PHP
- classe de conexão PDO
- demais arquivos necessários para executar o sistema

O trabalho é individual. O repositório deverá ser disponibilizado ao professor conforme orientação da disciplina.

16. Fluxo esperado



A sequência esperada é: entender o problema → modelar → criar o banco → inserir dados → escrever SQL → criar classes → aprender PDO → implementar CRUD → integrar ao front.